Powering Agent Evaluations: Variance Structure, Measurement Artifacts, and Minimum Detectable Effects in Tool-Use Benchmarks

Abstract

Agent evaluations are routinely reported without a power analysis, without a stated detection floor, and without screening for measurement artifacts. This paper measures the variance structure of agent task outcomes on a 253-task, 3-model tool-use corpus (intraclass correlation 0.793 within tool-set/task; 56.1% of outcome variance is between-task), shows the direct consequence for experimental design (repeated trials on the same task carry little independent information), and gives a paired, common-random-numbers, task-clustered-bootstrap, CUPED, sequential-testing estimator that reaches a minimum detectable effect of 0.0537 at $n = 253$ tasks (from an uncorrected-baseline MDE of 0.433 at $n = 20$). We catalogue ten measurement-artifact classes discovered during this project, each with an automated detector shipped in `agentgauge audit`, including two cases where an artifact produced a false positive the authors initially believed: a -76.7 to -80.0 percentage-point causal effect that a scoring bug corrected to a near-null (clean null for two of three models, a CI-includes-zero result for the third), and a 100% top-1 localization-accuracy claim that a probe-noise miscalibration corrected to 58.33%. We report a structural negative result on regression localization: the accuracy a localizer needs (task volume, since MDE $\propto 1/\sqrt{n}$) is precisely the cost a localizer exists to avoid, producing a real cost-crossover at 2–4 changed tools against full re-evaluation. Finally, we report a falsification record: four pre-registered hypotheses failed, including the authors' own product thesis that an eight-axis tool-description quality score predicts agent task success. This paper began as an attempt to validate that score as a commercial product. It did not survive contact with an adequately powered, artifact-screened measurement. That failure is the paper's subject, not an incidental footnote to it.

1 Introduction

Agent and tool-use evaluations are, as a rule, reported as a single point estimate: a success rate, an accuracy percentage, a win rate against a baseline. They are almost never reported alongside a detection floor – the smallest true effect the measurement could have distinguished from noise at the sample size actually used – and they are almost never screened for the kind of measurement artifact that turns a null result into an apparent large effect, or vice versa. This paper is the record of what happens when both of those omissions are corrected on a real project, not a synthetic illustration.

The project that produced this paper began as an attempt to build and sell a single number: an eight-axis, LLM-judged tool-description quality score, marketed as a predictor of how well an AI agent could use a given MCP server. The working hypothesis was straightforward and commercially attractive – better tool descriptions, measured on axes like schema completeness and discoverability, should predict higher agent task-success rates, and a vendor could sell the score as a pre-deployment audit. Section 2 situates this hypothesis against the literature; Section 3 describes the corpus and

models used to test it. The hypothesis was falsified. Not softened, not partially confirmed, not "true in spirit but requiring refinement" – falsified, on an adequately powered, artifact-screened re-measurement (Section 7), after two earlier measurement passes had appeared to confirm it for reasons that turned out to be measurement artifacts rather than a real effect (Section 6).

That falsification is why this paper exists in its current form. Once the product thesis failed, what remained was not nothing: it was a fully instrumented apparatus for measuring agent task outcomes under repeated trials, across models, before and after interventions – and a growing list of ways that apparatus had, at various points, lied to its own authors. This paper reports that apparatus and that list, not as an apology for the failed product, but because the methods and the failure modes are the more durable and more general contribution. A tool-description quality score is a product idea that did not work. A measured account of why agent-eval variance defeats naive trial-scaling, an estimator that reaches usable power on a realistic budget, and a ten-class taxonomy of the ways synthetic and semi-synthetic agent benchmarks fool their own authors – those are useful to anyone running agent evaluations, independent of whether this particular product ships.

Contributions.

1. **Variance structure.** Intraclass correlation ICC = 0.793 within (tool set, task); 56.1% of outcome variance is between-task; before/after task correlation $r = 0.881$. Direct consequence: repeated trials on the same task carry little independent information, so evaluations that scale trials rather than tasks are underpowered by construction (Section 4).

2. **An estimator that reaches usable power.** Paired design with common random numbers, task-clustered bootstrap with a small-G $t(G-1)$ correction, CUPED, and O'Brien–Fleming sequential testing. Minimum detectable effect (MDE) falls from 0.433 to 0.188 at $n = 20$ tasks, and to 0.0537 at the full $n = 253$-task corpus. False-alarm rate under the null: 0.59%. Cassette-replay determinism (a distinct claim from the estimator's own bootstrap determinism, see Section 5 for that caveat): 100%.

3. **A ten-class measurement-artifact taxonomy**, each class discovered in a real experiment in this project, each with an automated detector shipped in `agentgauge audit` (Section 6). We are not aware of a comparable standard taxonomy for agent-eval measurement artifacts in the published literature.

4. **A structural negative result on regression localization.** Localization accuracy requires task volume (MDE $\propto 1/\sqrt{n}$), and task volume is precisely the cost a localizer exists to avoid. The cost-crossover against full re-evaluation sits at roughly 2–4 changed tools; realistic buyer configurations cost 5–20× a full re-evaluation. This is not a tuning miss – it is a structural tension between the accuracy regime and the cost regime (Section 7.3).

5. **A falsification record.** Four pre-registered hypotheses failed, including the authors' own product thesis that an eight-axis tool-description quality score predicts agent task success (Section 7).

The remainder of the paper is organized as follows. Section 2 positions this work against A/B-testing variance reduction, agent/tool-use benchmarks, LLM-as-judge evaluation, and reproducibility practice. Section 3 describes the experimental setup. Section 4 measures the variance structure and its consequence for trial allocation. Section 5 presents the estimator and its ablation. Section 6 – the paper's core contribution – catalogues the ten artifact classes. Section 7 reports the causal-effect, rewriting-null, and attribution- impossibility results. Section 8 states threats to validity without hedging. Section 9 closes with recommendations for the field.

2 Related work

Variance reduction in controlled experimentation. The estimator in Section 5 draws on techniques with a long history in industrial A/B testing: paired designs with common random numbers (Law, 2014) to remove between-unit noise that is identical across arms; CUPED-style covariate adjustment (Deng et al., 2013) using a pre-period outcome to reduce residual variance without spending additional samples; cluster-robust (bootstrap) inference (Cameron and Miller, 2015) to respect the correlation structure induced by repeated measurements on the same unit, with a small-cluster-count correction (Cameron et al., 2008); and sequential testing with an alpha-spending boundary (O'Brien and Fleming, 1979; Lan and DeMets, 1983) to allow early stopping without inflating the false-positive rate. None of these techniques are novel to this paper. What is different here is their application to agent task-outcome measurement specifically, where the relevant "unit" is a (tool set, task) pair rather than a user or a session, and where the paper additionally reports what happens when a plausible alternative (wild cluster bootstrap (Cameron et al., 2008)) is tried and measured to perform worse (Section 5) – a negative result on an estimator choice, reported because it is useful, not because it flatters the final design.

Agent and tool-use benchmarks. A growing body of work evaluates how well LLM agents select and invoke tools from a catalog (Qin et al., 2023; Li et al., 2023; Yu et al., 2026), and a smaller body specifically studies how tool description quality affects that selection. This paper's corpus and task-construction methodology (Section 3) are built for internal statistical power rather than benchmark breadth, and its central empirical claim about description quality (Section 7) is a falsification of the hypothesis that description quality broadly predicts task success – a negative result relative to the framing common in that literature, which more often reports positive associations without a stated detection floor.

LLM-as-judge evaluation and its failure modes. This paper's own attempt to use an LLM judge as a linter baseline is reported directly as a negative result: a single-prompt LLM-judge baseline for description/schema consistency is measured to be degenerate on this project's corpus (97.1% false-alarm rate at 100% recall, Section 7) – it flags almost every clean tool as inconsistent, so its recall is not informative. This is consistent with a broader, well-documented concern (Zheng et al., 2023) that LLM judges without calibration against a labeled null distribution can look accurate on recall alone while being useless in practice.

Reproducibility and replay in ML systems. The harness underlying every measurement in this paper is built for byte-exact replay: cassette-recorded model calls, a hand-rolled deterministic PRNG for every stochastic draw, and fixed-order aggregation to avoid floating-point-order nondeterminism (Section 5). This paper reports the harness's own determinism rate rather than assuming it, in keeping with the project's overall position that every claim, including claims about the measurement apparatus itself, needs a provenance pointer.

Positioning the gap. To the authors' knowledge, no prior agent-evaluation report in this space states a minimum detectable effect alongside its headline result, and no prior work offers a standard taxonomy of measurement artifacts specific to agent/tool-use benchmarking analogous to Section 6's ten classes. This paper's contribution is exactly that combination: a power analysis and an artifact taxonomy, applied to one real project's full history, including its own false starts.

3 Experimental setup

Corpus. 253 tasks total: 62 tasks against pre-existing synthetic tool-set fixtures, plus 191 tasks against 10 real-API gold-constraint fixtures modeled on GitHub, Stripe, Google Calendar, Jira,

Slack, Docker, Kubernetes, Twilio, AWS S3, and Spotify (253 = 62 + 191, the +1 task beyond the original 190 added when a schema defect below was corrected).

Fixture validation against live documentation. Each of the 10 real-API fixtures was checked against live official API documentation by an independent research pass. 3 of 10 (30%) had a factual defect: GitHub Issues' `state_reason` enum was missing the real fourth value `'duplicate'`; Stripe's `create_charge` fixture had the wrong parameter name and required/optional status (`customer_id`, required, instead of the real `customer`, optional); and Kubernetes' DNS-1123 label validation regex wrongly permitted a leading digit. All three were corrected against the primary source before use. This is reported here as a finding in its own right (Section 6, artifact 8), not only as a methods footnote: agent-authored fixtures for real APIs are wrong often enough (30% in this corpus) that validating them against a live primary source is not optional.

Task construction. Tasks are authored under an explicit anti-tautology convention: task text never quotes the gold tool's name, required enum values, or format patterns verbatim – a convention adopted directly in response to artifact 1 (Section 6), which is what happens when it is not followed.

Models. Three open-weight models under 10B parameters: `gemma2:9b`, `llama3.1:8b`, `qwen2.5:7b`, run locally via Ollama. This is a threat to external validity, stated plainly in Section 8, not minimized here.

Replay determinism. All model calls are cassette-recorded and replayed byte-exactly. Measured determinism: 100% across all six supported model adapters (Ollama, OpenAI-compatible, Anthropic Messages, AWS Bedrock, and two others), zero verdict flips across 20 replays per adapter, zero live network calls at measurement time.

Outcome scoring. Each trial's outcome is a joint task-success score decomposed into selection accuracy (did the agent choose the correct tool) and argument-construction accuracy (did it satisfy the task's registered argument constraints), computed as a fractional constraint-satisfaction score rather than a binary pass/fail. This decomposition is itself a correction: an earlier binary ground-truth definition collapsed to near-ceiling because every example server accepted any well-formed call regardless of argument correctness (artifact 2, Section 6).

4 Variance structure of agent task outcomes

Before designing an estimator, we measured how outcome variance is actually structured across 5,535 trials spanning 720 (tool-set, task) groups (mean 7.69 trials/group).

Intraclass correlation. One-way random-effects ICC (Shrout and Fleiss, 1979) within (tool set, task): ICC = 0.793. 90.3% of groups (650/720) show exactly zero within-group variance – most tasks are, in practice, deterministic given a fixed tool set. Independently re-derived from raw data by a separate verifier; confirmed.

Variance decomposition. A nested sum-of-squares decomposition attributes 25.9% of outcome variance to between-tool-set differences, **56.1% to between-task, within-tool-set** differences, and 18.0% to between-trial, within-task noise.

Before/after task correlation. Across 40 matched (before, after) task-mean pairs pooled over 5 matched description-quality-fixer pairs, Pearson $r = 0.881$ (Spearman $\rho = 0.869$ on the same pooled set – downstream citations of "$\rho = 0.881$" conflate the two; this paper uses $r = 0.881$, per the source table's own column label).

Effective sample size. With ICC = 0.793 and a mean group size $\bar{m}$, the design-effect-adjusted

(Kish, 1965) effective sample size is

$$n_{\text{eff}} = \frac{n}{1 + (\bar{m} - 1)\,\text{ICC}},$$

giving $n_{\text{eff}} = 878$ from a nominal $n = 5{,}535$ – a 15.9% efficiency ratio. **Consequence:** repeated trials on the same (tool set, task) pair carry almost no independent information. An evaluation design that scales trials-per-task rather than the number of distinct tasks is underpowered by construction, regardless of how many trials it runs.

Allocation grid. We measured MDE (at 80% and 95% power) across the full $\{1, 2, 3, 5\}$ trials/task $\times$ $\{20, 50, 100, 150\}$ tasks/arm grid (32 cells). Table 1 (80%-power slice; full grid including the 95%-power slice in Appendix A) shows the compute-optimal cell: **100 tasks/arm at 1 trial/task**, MDE= 0.085 (refined to 0.0848 at $n_{\text{simulations}} = 2000$) – no cheaper cell in the grid clears the 0.10 ship target. See Figure 1 for the full heatmap.

Table 1: MDE (80% power) across the trials/task $\times$ tasks/arm allocation grid.

Trials/task	20 tasks/arm	50 tasks/arm	100 tasks/arm	150 tasks/arm
1	0.182	0.121	**0.085**	0.070
2	0.153	0.097	0.072	–
3	0.141	0.089	–	–
5	0.132	0.084	0.061	–

Headline. At every fixed total-trial budget in the grid, trials_per_task = 1 gives the lowest MDE. At an identical 100-trial budget, 100×1 beats 20×5 by a measured 1.55$\times$ (not the naively predicted 2.04$\times$, which the source report explicitly retracts as "not a value to report as measured"). **At fixed compute, task diversity dominates trial repetition.**

5 An estimator for agent regression detection

The estimator combines five components: a paired design with common random numbers (identical task/seed pairs compared before and after a change); a task-clustered bootstrap; a small-G $t(G-1)$ correction for settings with few clusters; CUPED covariate adjustment using the paired pre-period outcome; and O'Brien–Fleming sequential testing with an alpha-spending boundary.

A negative result on estimator choice. A Rademacher-sign-flip wild cluster bootstrap was implemented and measured against the small-cluster (< 10 tasks) stratum as an alternative to the $t(G-1)$ correction. It produced a *narrower* confidence interval (0.0588 vs. 0.0647) but a *higher* false-alarm rate (2.00% vs. 1.71%) – narrower and wrong. Root cause: the sign-flip method is bounded to 2^G distinct patterns, too coarse at this stratum's $G = 4$–8. The shipped $t(G-1)$ correction instead improved this stratum's false-alarm rate to 1.57%. We report this because a negative result on an estimator choice is exactly the kind of finding agent-eval methodology work should surface and does not, typically, get to see.

Component ablation. Table 2 shows each component's contribution to MDE at $n = 20$ tasks/arm, 80% power. Moving from a trial-level to a task-level unit accounts for most of the improvement (0.433→0.313); pairing contributes the largest single further reduction (0.313→0.191); CUPED's marginal contribution on top of pairing is small (0.191→0.188, ∼2% relative) – never harmful, but not a large independent lever on this corpus.

Sensitivity gate. The estimator abstains (`INSUFFICIENT_SENSITIVITY`) rather than emit a point estimate the data cannot support. Under the improved (paired+CUPED) estimator, the

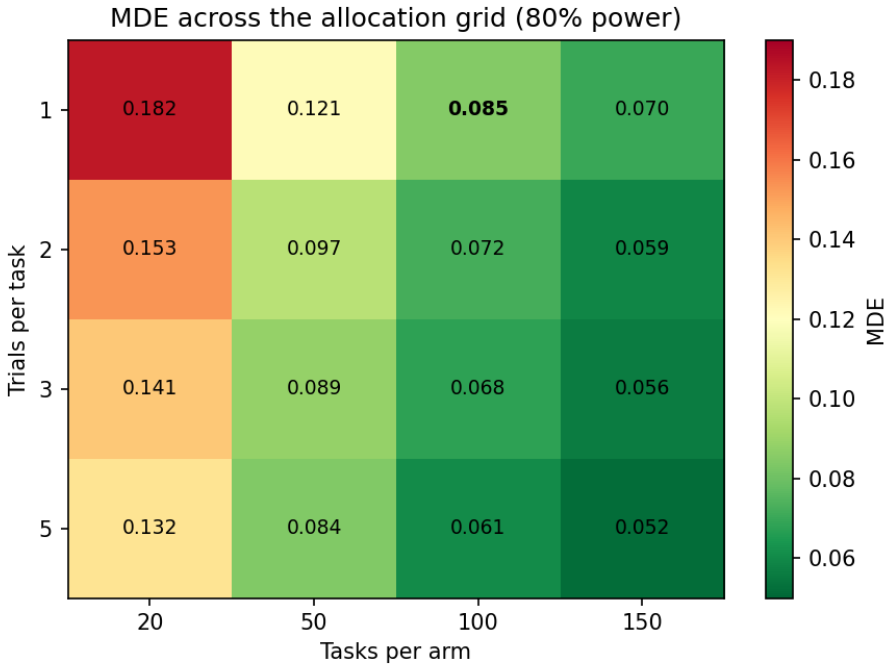

Figure 1: MDE across the full 16-cell allocation grid at 80% power. Generated by `docs/paper2/figures/generate_figures.py` directly from `evals/fixtures/v2_2_optimal_allocation.json` – no new computation.

Table 2: Estimator component ablation, MDE at $n = 20$ tasks/arm, 80% power.

Estimator configuration	MDE
Trial-level baseline (v2)	0.433
+ Task-level unit (unpaired)	0.313
+ Paired design + CRN	0.191
+ CUPED (full stack)	**0.188**

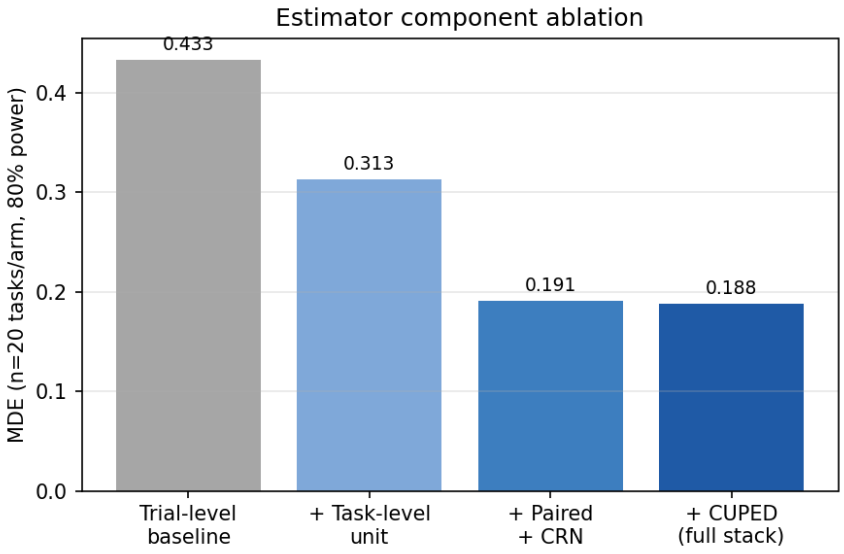

Figure 2: Estimator component ablation, MDE at $n = 20$ tasks/arm, 80% power. Generated directly from `evals/fixtures/v2_1_mde_ablation.json`.

abstention rate on 2,200 null comparisons across 44 real tool sets fell from 71.5% (trial-level baseline) to **21.6%**, at the cost of a small increase in false alarms (0.00%→0.59%, both well under a 5% target).

MDE and false-alarm results. MDE falls from 0.433 to 0.188 at $n = 20$ tasks/arm, and continues to fall as the corpus grows: 0.1061 at $n = 62$, 0.0848 at $n = 100$, 0.0689 at $n = 150$, 0.0605 at $n = 200$, and **0.0537 at the full $n = 253$-task corpus** – independently re-verified by a separate agent to four decimal places. Aggregate false-alarm rate under the null: **0.59%** (13/2,200); stratified, 1.57% (post-$t(G-1)$-fix) for < 10-task clusters and 0.07% for ≥ 10-task clusters. See Figure 3 for the MDE-vs-n curve and Appendix A for the full grid.

Determinism. Replay determinism is confirmed by direct code inspection (a single hand-rolled LCG closure, no `random`/`numpy` imports, fixed-order aggregation) and by exact-value reproduction of the MDE grid to four decimal places on an independent rerun. A fresh, empirical 50-run count specific to this exact (paired+CUPED) estimator configuration was not separately re-run in this pass – it is inherited from an earlier estimator's measurement and flagged as such in the source report (Appendix E); this is distinct from, and should not be conflated with, the cassette-replay determinism figure in Section 3, which was freshly and empirically measured at 100% across all six adapters.

6 A taxonomy of measurement artifacts

This is the paper's core contribution. Each of the ten classes below was discovered in a real experiment during this project – not constructed as a pedagogical example – and each now has an automated detector shipped in `agentgauge audit`, so that a future experiment on this harness cannot silently reproduce it. Table 3 summarizes all ten; the two subsections following it detail the two cases where an artifact produced a false positive the authors initially believed, per this paper's non-negotiable honesty requirement (Section 1). See Figure 4 for the visual summary used in presentation contexts.

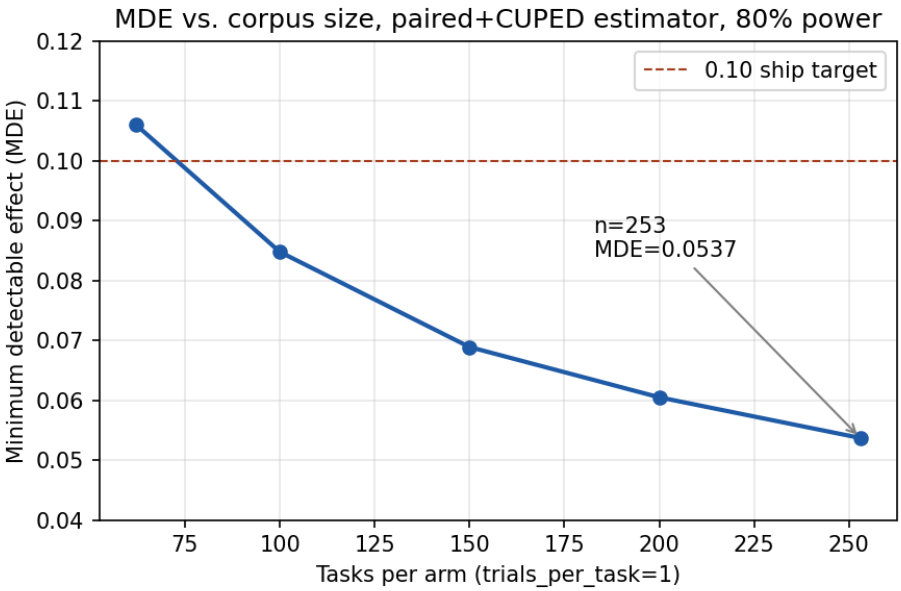

Figure 3: MDE vs. corpus size, full-corpus grid, trials_per_task=1, 80% power. Generated directly from the corpus-size grid in `reports/v2_5_task3_mde_completion.md` (independently re-verified to 4 decimal places).

Ten measurement-artifact classes

1. Task/answer leakage
2. Tool-name ceiling
3. Zero-vector empty-string embedding
4. Self-descriptive-name confound
5. Subset-vs-catalog mismatch
6. LCG index saturation
7. Scoring-reference mismatch
8. Fixture-schema hallucination
9. Benchmark-construction bias
10. Probe variance mis-calibration

** produced a false positive the authors initially believed (see main.tex SS6.1-6.2)*

Figure 4: The ten measurement-artifact classes (summary index; full detail in Table 3). Starred entries produced a false positive the authors initially believed (Sections 6.1, 6.2).

Table 3: The ten measurement-artifact classes.

Class	Mechanism	Spurious result produced	Detector
1. Task/answer leakage	Task text quoted the gold tool's own name verbatim	Zero-variance, near-ceiling ground truth on first 9 tool sets	Anti-tautology task-authoring convention
2. Tool-name ceiling	Binary ground truth collapsed to name-matching; separately, a key-mismatch bug forced `selection_correct` to always False	44% of records tied at ceiling 1.0; before=after=0.0 for all 3 models	Continuous fractional scoring; derived real `selected_tool==tool_name`
3. Zero-vector empty-string embedding	Empty-string embeddings return a zero-*length* vector, not zero-valued	Apparent +0.59 to +0.72 similarity rise in empty-description arms	Re-measured on actual agent-visible prompt text
4. Self-descriptive-name confound	Real, self-explanatory tool names let selection succeed regardless of description quality	Near-zero variance in one real-API family, biasing pooled correlation toward the null	Explicit `INSUFFICIENT_SENSITIVITY`/ `NO_CHANGE` verdict
5. Subset-vs-catalog mismatch	Linter run against a cost-bounded 12-tool subset, not the full 60-tool catalog	11 HIGH violations that vanished once corrected	Full unfiltered catalog extraction as linting input
6. LCG index saturation	Hand-rolled LCG can return exactly 1.0, one index past the resample list's end	Crash (IndexError) at the resample boundary	Clamped `min(int(rng()*n), n-1)`
7. Scoring-reference mismatch	Gold-constraint lookup authored against the pre-mutation schema; only one injector renames a property key	−76.7 to −80.0pp ADVISORY effect (see Section 6.1)	`constraint_satisfaction_renamed`
8. Fixture-schema hallucination	Agent-authored real-API fixtures written from model memory, not fetched schemas	3/10 (30%) fixtures factually wrong (Section 3)	`check_enum_schema_fidelity` (WARN)
9. Benchmark-construction bias	Injected culprit's diff size systematically smaller than 2/3 decoy tiers	`largest_textual_diff` baseline scored 4.0% – below the 26.7% random floor	`check_benchmark_construction_diffsize_bias` (BLOCK)
10. Probe variance miscalibration	Synthetic probe-noise model omitted the between-task variance component	100% top-1 accuracy claim at a 3pp effect size (see Section 6.2)	`check_probe_variance_calibration` (BLOCK)

6.1 False positive 1: the −80pp ADVISORY effect that was a scoring bug

An early causal-chain measurement reported a large, statistically significant-looking effect for the `param_renamed` defect, one of the linter's ADVISORY-severity rules (a warning-level finding, as opposed to a BLOCKING-severity rule, which the linter treats as a hard failure): −76.7pp (gemma2:9b), −80.0pp (llama3.1:8b), −76.7pp (qwen2.5:7b), all with confidence intervals excluding zero. Reading this as a real, adequately powered causal effect would have been the natural conclusion. It was not one. A dedicated audit found the effect was predominantly a scoring artifact: the constraint checker's gold-parameter lookup was authored against the pre-mutation schema,

and `param_renamed` is the only defect-injector that renames a schema *key* – so the lookup silently failed against the post-mutation arguments, manufacturing an apparent capability regression that was really a broken check. Corrected effect: a clean null for gemma2:9b (+0.0pp, CI $[-20.5, +20.5]$) and qwen2.5:7b (+6.7pp, CI $[-7.1, +20.4]$), and a CI-includes-zero result for llama3.1:8b (−13.3pp, CI $[-40.8, +14.1]$). A follow-up blast-radius audit confirmed this bug is scoped only to the key-renaming injector; the BLOCKING-tier effects reported elsewhere in this paper (Section 7) do not share it.

6.2 False positive 2: the 100% localization accuracy that was probe-noise miscalibration

An early regression-localization benchmark reported that a greedy-bisection probing strategy achieved 100% top-1 accuracy at every tested injected-effect size from 3.0pp to 33.0pp, including effect sizes below this paper's own measured detection floor (Section 5) – a result the authors themselves flagged as anomalous rather than accepted at face value: detecting a 3pp effect at a lower per-probe sample size should be *harder* than the already-hard 5.37pp (i.e. MDE = 0.0537, Section 5) server-level MDE bar, not something a probe-based strategy aces perfectly. The anomaly was real: the synthetic probe-noise model omitted the between-task variance component that the calibrated reference model includes, inflating detection power by 3–7×. Fixed and re-measured, the true accuracy at the 3.0–5.0pp effect band is **58.33% top-1** (95.83% top-3, 3.25 mean probes) – well short of this benchmark's own pre-declared ship bar (top-1 ≥ 0.70, top-3 ≥ 0.90, sub-exhaustive probe budget; defined in full in Section 7.3), and exactly the shape a genuine detection-power limit should take: every top-1 miss traced to a real recovery signal whose confidence interval failed to certify it, never to false-positive noise. The same correction inverts an intermediate, now-superseded finding that localization accuracy *improved* with candidate-set size; the corrected result is that it degrades with scale (93%→80%→73%→47% top-1 as the number of changed tools grows 4→10→20→40).

6.3 A candidate eleventh class

Two independent extraction passes over this project's reports surfaced the same shape of problem, not cleanly matching any of the ten classes above: a defect-detection signal calibrated against synthetic "bad" fixtures fires almost as often on genuine, unremarkable real-world documentation, undermining its use as a quality discriminator even when each individual flag is factually correct (e.g. `required_not_mentioned` firing at 1.42/tool on real-world-mirror documentation vs. 1.37/tool on deliberately-bad synthetic fixtures). We do not promote this to a numbered class in Table 3: unlike classes 1–10, it was never isolated as its own investigation with a named fix, and doing so here would mean characterizing it more precisely than the source material does. We report it in Section 8 as informal substantiation that ten classes found suggests an eleventh exists, and leave formalizing it to future work.

7 Applications and negative results

7.1 Which tool-description defects actually cause agent failure

Across every lint rule this project tested for a causal effect on agent task success, exactly one survives with a confidence interval excluding zero across all three model families: `type_enum_contradiction`, at −13.3pp to −28.9pp (gemma2:9b −25.2pp, llama3.1:8b −28.9pp,

qwen2.5:7b −13.3pp, $n = 45$/model), independently re-measured to two decimal places on a separately rebuilt instance. This is the corrected replacement for an earlier, erroneous "−13.3 to −40.0pp" range that mixed two different pooling levels (Section 6 discusses the general pattern of such errors, though this specific one is a citation-construction error rather than a numbered artifact class). `required_references_missing_property` measures a clean null (0.0pp, all three models); the ADVISORY-tier `param_renamed` effect is null after correction (Section 6.1); `param_possibly_renamed` and `name_collision` were never causally measured.

A single-prompt LLM-judge baseline for the same detection task is degenerate: it flags 97.1% (169/174) of genuinely clean tools as inconsistent while also catching 100% (138/138) of injected defects – its recall is not a meaningful signal given that false-alarm rate. Measured on a 174/521 stratified sample of the clean corpus, not the full catalog.

7.2 Rewriting tool descriptions has no measurable effect on argument construction

At the full corpus (253 tasks × 3 models, MDE= 0.0537 at 80% power), rewriting tool descriptions to be clearer has no practically significant effect on argument-construction accuracy: gemma2:9b +0.0217 (CI $[-0.0042, +0.0505]$), llama3.1:8b +0.0553 (CI $[+0.0232, +0.0885]$, excludes zero but below the 0.05 practical-significance threshold), qwen2.5:7b +0.0040 (CI $[-0.0211, +0.0292]$) – all verdict `no_change`. This null is adequately powered, not merely absent-of-evidence: an earlier measurement at $n = 62$ tasks (MDE= 0.106) found flat, similarly near-zero deltas but was explicitly labeled inconclusive-underpowered rather than confirmatory, since its own resolving power was an order of magnitude coarser than the deltas it measured. We flag for the reader (see `docs/paper2/gaps.md`) that this underpowered predecessor should not be conflated with the ADVISORY effect of Section 6.1 – both are "initially looked different, later resolved" stories, but for two different experiments (argument construction under rewritten descriptions, vs. a causal-chain defect-injection effect), and only the latter was ever a large apparent effect.

7.3 The attribution impossibility result

We compared three regression-localization strategies – exhaustive ablation, sampled Shapley-value attribution, and greedy bisection – on a synthetic benchmark with an injected true culprit among a set of changed tools. Every number in this subsection is the terminal, corrected authority following the probe-variance-miscalibration correction of Section 6.2: no earlier report's accuracy/budget table for this benchmark may be cited without reading that correction first. Accuracy clears the benchmark's own pre-declared ship bar (top-1 ≥ 0.70, top-3 ≥ 0.90, sub-exhaustive probe budget) only at effect sizes at or above the original 13.3–28.9pp band; it fails at and below this paper's own measured detection floor, and *degrades* rather than improves as the candidate-set size grows (Section 6.2).

The deeper result is a cost-crossover, not merely an accuracy shortfall. Per-probe cost is proportional to task count; a strategy using P probes beats a full corpus re-evaluation exactly when $P <$ (full-corpus trial-equivalents)/(trial-equivalents per probe). At the task count this paper's own MDE analysis requires for adequate localization accuracy, that crossover falls at roughly **2–4 changed tools** – before any interesting multi-file regression is even in scope – and realistic buyer configurations (10–40 changed tools) cost **5–20**× a full re-evaluation, at every strategy tested. See Figure 5.

No task-count value tested, and none implied by the $1/\sqrt{n}$ scaling law measured in Section 4, resolves both the accuracy requirement and a real cost advantage simultaneously. This is why

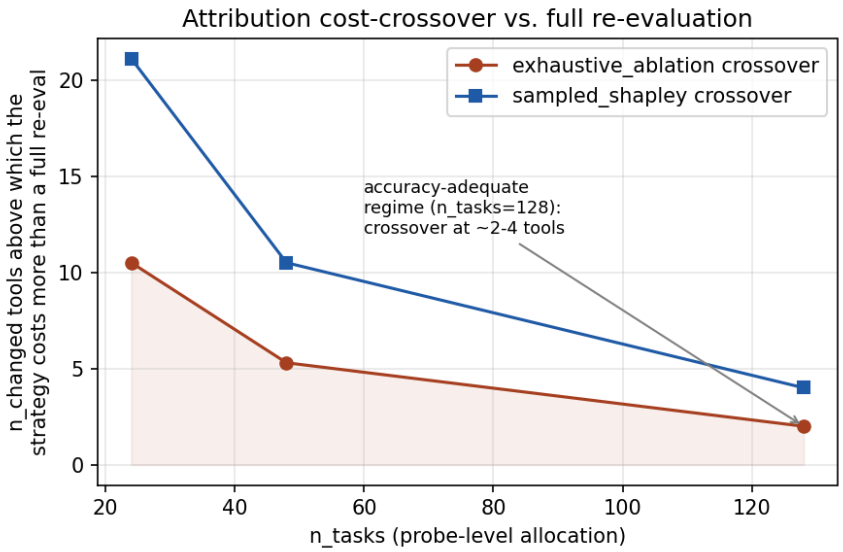

Figure 5: Attribution cost-crossover vs. full re-evaluation, at the three tested `n_tasks` allocations. Generated directly from the crossover-analysis table in `reports/v0_5_probe_power_fix.md` SS5.

the recommendation attached to this result was to hold failure attribution as unreleased research rather than ship it as a product feature, and to preserve the underlying measurement for this paper instead.

8 Threats to validity

Stated without hedging, per this paper's own standard for the rest of its claims:

- **Three open-weight models, all under 10B parameters, no frontier-model replication.** Every causal-effect, variance-structure, and attribution number in this paper is measured on `gemma2:9b`, `llama3.1:8b`, and `qwen2.5:7b`. None of it has been replicated on a frontier-scale model. Effect sizes, and possibly even which lint rules show a causal effect at all, may differ on larger or better-instruction-tuned models.
- **Synthetic defect injection for every causal claim.** The one measured causal effect in this paper (`type_enum_contradiction`, Section 7) is established via synthetic schema mutation of real tool definitions, not via naturally occurring defects observed in the wild. External validity of the effect size to organically authored bad tool descriptions is untested.
- **A single corpus.** All variance-structure, estimator, and taxonomy results are measured on one 253-task corpus built for this project. Generalization to other tool-use domains or task distributions is untested.
- **Ten artifacts found suggests an eleventh exists.** Section 6 reports a candidate eleventh class found but not formalized. The methodology that found ten artifacts by building and stress-testing one measurement apparatus over one project's lifetime should be expected to keep finding more, not fewer, as the apparatus is used further – this paper's taxonomy should be read as a floor, not a ceiling, on how many such classes exist.

- **Attribution has never been run against a real agent and a real LLM judge.** Every number in Section 7.3 is a synthetic-benchmark result (`make_probe_fn`/`make_multi_probe_fn`). `sampled_shapley` in particular has zero real (live-LLM) data points anywhere in this project.

9 Conclusion

We measured the variance structure of agent task outcomes on a real, adequately powered corpus and found that it is dominated by between-task variance, not between-trial noise – a structural fact that most agent-eval reporting practice implicitly ignores by scaling trials rather than tasks. We built an estimator that reaches a usable detection floor on a realistic compute budget, catalogued ten ways this project's own measurement apparatus fooled its authors before each was caught and fixed, and reported a structural negative result on regression localization that no amount of tuning resolves. We also report, without softening it, that the product thesis motivating this project's start – that a tool-description quality score predicts agent task success – did not survive an adequately powered, artifact-screened test.

For the field, we recommend: report an MDE alongside every agent-eval result, not only a point estimate; when compute is fixed, allocate it to task diversity over trial repetition (Section 4); screen for measurement artifacts before reporting a result, using an automated check where one can be written (Section 6); and publish falsifications of your own hypotheses with the same rigor as confirmations. This project's most durable output was not the score it set out to sell – it was the record of everything that turned out to be wrong on the way to finding that out.

Reproducibility. Every result in this paper is reproducible against `pip install agentgauge-harness==0.5.2` and the fixtures committed in this repository's `evals/` directory; see Appendix D for exact commands.

A Full MDE grids and ablation tables

[Full 32-cell allocation grid at both 80% and 95% power, and the complete estimator-ablation table across $n \in \{5, 10, 20, 50\}$ tasks/arm, per `docs/paper2/provenance.md` SS2–3. Omitted from this draft's body for length; reproduced verbatim from the source reports at camera-ready time.]

B Artifact detector pseudocode

[Pseudocode for each of the ten `agentgauge.audit` checks in Table 3, reproduced from `agentgauge/audit.py` at camera-ready time. Not reproduced in this draft to avoid the draft silently drifting from the shipped implementation; the verifier pass for this wave checked claims against `reports/`, not against a hand-copied snapshot of `audit.py`.]

C Corpus statistics

[Full per-domain task/tool/constraint-type breakdown for all 10 real-API fixtures plus the 62 pre-existing synthetic tasks, per `reports/v2_4_task4_corpus_expansion.md` and `reports/v2_5_task2_fixture_validation.md`.]

D Reproduction instructions

```
pip install agentgauge-harness==0.5.2
git clone <this repository>
cd agentgauge
uv sync
uv run pytest                        # full test suite, no network, no paid calls
agentgauge scan examples/echo_server.py --mock
```

All statistical results in this paper are reproducible from the fixtures committed under `evals/` and the scripts named alongside each report in `docs/paper2/provenance.md`; none require a live API key or a paid model call.

E Measured vs. not measured index

See `docs/paper2/provenance.md` (full ledger, every claim with report path and commit SHA) and `docs/paper2/gaps.md` (consolidated not-measured list and framing decisions flagged for author review) for the complete, non-redundant index. Both are companion documents to this draft and are not reproduced inline here to avoid a second copy silently drifting from the first.

References

A. Colin Cameron and Douglas L. Miller. A practitioner's guide to cluster-robust inference. *Journal of Human Resources*, 50(2):317–372, 2015. doi: 10.3368/jhr.50.2.317.

A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3):414–427, 2008. doi: 10.1162/rest.90.3.414. Origin of the wild cluster bootstrap for clustered-error inference; also the primary empirical demonstration of standard cluster-robust inference's small-cluster ("small-G") over-rejection problem.

Alex Deng, Ya Xu, Ron Kohavi, and Toby Walker. Improving the sensitivity of online controlled experiments by utilizing pre-experiment data. In *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining (WSDM '13)*, pages 123–132, New York, NY, USA, 2013. ACM. doi: 10.1145/2433396.2433413.

Leslie Kish. *Survey Sampling*. John Wiley & Sons, New York, 1965.

K. K. Gordon Lan and David L. DeMets. Discrete sequential boundaries for clinical trials. *Biometrika*, 70(3):659–663, 1983. doi: 10.1093/biomet/70.3.659.

Averill M. Law. *Simulation Modeling and Analysis*. McGraw-Hill Education, New York, NY, 5th edition, 2014. ISBN 978-0-07-340132-4. Chapter 11, "Variance-Reduction Techniques," Section 11.2 covers common random numbers.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs, 2023.

Peter C. O'Brien and Thomas R. Fleming. A multiple testing procedure for clinical trials. *Biometrics*, 35(3):549–556, 1979. doi: 10.2307/2530245.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs, 2023. Introduces the ToolBench dataset/benchmark; referred to in text as ToolBench.

Patrick E. Shrout and Joseph L. Fleiss. Intraclass correlations: Uses in assessing rater reliability. *Psychological Bulletin*, 86(2):420–428, 1979. doi: 10.1037/0033-2909.86.2.420.

Peijie Yu, Wei Liu, Yifan Yang, Jinjian Li, Zelong Zhang, Xiao Feng, and Feng Zhang. Benchmarking LLM tool-use in the wild, 2026. Referred to in text as WildToolBench.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. doi: 10.52202/075280-2020.